

Análise Comparativa SSSP: Dijkstra, Bellman-Ford e BMSSP

Quebrando a Barreira de Ordenação em Grafos

Dupla A: Algoritmos — **Dupla B:** Engenharia — **Dupla C:** Documentação

Universidade Federal de Alfenas – UNIFAL-MG

4 de maio de 2026

Sumário

- 1 Introdução e Objetivos
- 2 Estrutura Matricial da Equipe
- 3 Fundamentação Teórica: Algoritmo BMSSP
- 4 Implementação Computacional
- 5 Conclusões

O Problema (SSSP)

Dada uma origem única, determinar a distância mínima para todos os outros vértices do grafo. Algoritmos clássicos enfrentam uma **barreira de ordenação** de vértices.

Objetivos deste Projeto (TP 03):

- Implementar o Dijkstra com *Min-Heap* iterativo.
- Implementar o *Bellman-Ford* como *baseline* incondicional.
- Reproduzir e avaliar a complexidade $O(m \log^{2/3} n)$ do algoritmo recursivo Bounded Multi-Source Shortest Path (**BMSSP**) de *Duan et al. (2023)*.
- Criar um pipeline de coleta empírica N vs. Tempo de Execução.

Governança e Estrutura da Equipe (Matricial)

Para evitar silos de desenvolvimento, 6 membros atuarão em 3 duplas (**Cross-Functional**):

Dupla A (Core C)

Foco: Corretude do Dijkstra e BMSSP.

Revisão cruzada pela Dupla B.

Dupla B (Python Pipeline)

Foco: Gerador Erdos-Renyi e Orquestração (`make`).

Revisão cruzada pela Dupla C.

Dupla C (LaTeX & Relatoria)

Foco: Artigo final, Beamer e conformidade bibliográfica.

Revisão cruzada pela Dupla A.

Vantagem de Ponta a Ponta: Qualquer membro consegue simular e analisar todo o repositório em sua máquina através do comando único `make all`, trocando de papéis dinamicamente.

O Algoritmo BMSSP (Duan et al., 2023)

A referência do algoritmo BMSSP quebra a barreira de ordenação (Dijkstra restrito a $O(m \log n)$) através de particionamento e recursividade.

- **Amostragem Estocástica:** O conjunto ativo S é comprimido em um menor conjunto de pivôs P via `FindPivots`, que explora as bordas.
- **Limites Superiores (B):** O algoritmo estipula bounds limitantes e atende a chamadas menores recursivas (nível l da árvore decresce até o caso base onde atua o Dijkstra simples).
- **Custo Analítico:** O particionamento em pivôs e limites evita varrer a fila inteira para achar o mínimo absoluto. Resulta em tempo dominante dominado pela profundidade $l = \lceil \frac{\log n}{t} \rceil$, chegando a $O(m \log^{2/3} n)$.

Pseudocódigo: BMSSP (Duan et al.)

Algorithm 1 BMSSP Simplificado

```
1: procedure BMSSP( $G, S, B, l$ )
2:   if  $l == 0$  ou  $S = \emptyset$  then                                ▷ Caso base: Alcançou o fundo da recursão
3:     Dijkstra em  $G$  com origens  $S$  até a distância limite  $B$ 
4:   else
5:      $k \leftarrow \lfloor \log^{1/3} n \rfloor$                                 ▷ Fator de compressão
6:      $P \leftarrow \text{FINDPIVOTS}(S, k)$                                 ▷ Amostra  $\approx |S|/k$  pivôs
7:     BMSSP( $G, P, B, l - 1$ )                                       ▷ Busca rápida guiada por pivôs
8:     BMSSP( $G, S, B, l - 1$ )                                       ▷ Busca refinada com conjunto completo
9:   end if
10: end procedure
```

Impacto: Ao focar em P , a técnica relaxa iterativamente os bounds (B) da árvore de busca, quebrando o gargalo teórico de $O(m \log n)$ inerente à ordenação irrestrita.

Automação da Instanciação e Perfilometria

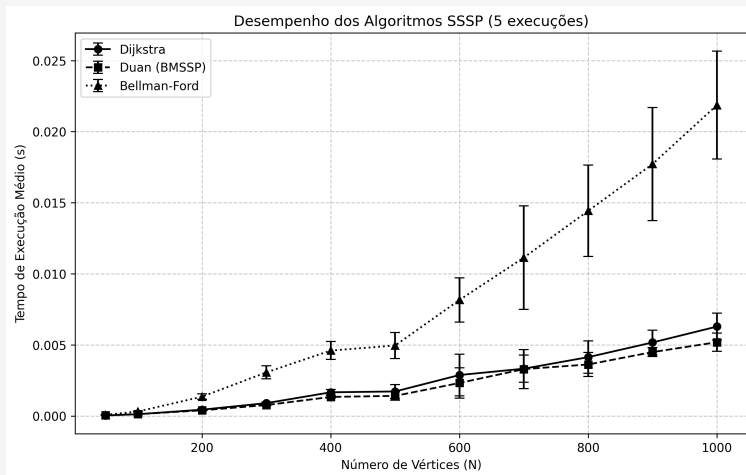
O desenvolvimento em C foi conectado a scripts em Python para uma análise empírica massiva:

Execução Pipeline (Zero-Noise)

- ❶ **Gerador de Instâncias:** A partir do `networkx`, grafos variados ($N \in [50, 1000]$) são passados pela entrada padrão ao binário C.
- ❷ **Execução e Extração:** O tempo processual (perfilometria via `clock()`) é coletado para os 3 algoritmos SSSP concorrentes.
- ❸ **Renderização Gráfica:** Um gráfico é embutido dinamicamente no relatório SBC.

A implementação precisou domar as estruturas de **ponteiros**, **Min-Heaps** e **memória** do BMSSP, garantindo que os subconjuntos de pivôs e matrizes de adjacência coubessem na memória de cache L2/L3.

Resultados Empíricos e Gráficos



- **Dijkstra Clássico** (Min-Heap): Desempenho hiper-otimizado com constante estrutural quase nula.
- **BMSSP (Duan)**: Vantagem teórica penalizada pelo *overhead* de chamadas recursivas, alocação de conjuntos e FindPivots em instâncias menores ($N \leq 1000$).

Resultados Estatísticos (5 Execuções Independentes)

Tabela: Tempo de Execução (média $\pm$ desvio padrão em segundos) para 5 execuções independentes.

Vértices (N)	Dijkstra	BMSSP (Duan)	Bellman-Ford (Outro)
50	0.0000 $\pm$ 0.0000	0.0001 $\pm$ 0.0000	0.0001 $\pm$ 0.0000
100	0.0001 $\pm$ 0.0000	0.0001 $\pm$ 0.0000	0.0003 $\pm$ 0.0000
200	0.0004 $\pm$ 0.0000	0.0004 $\pm$ 0.0000	0.0014 $\pm$ 0.0002
300	0.0009 $\pm$ 0.0001	0.0008 $\pm$ 0.0001	0.0031 $\pm$ 0.0005
400	0.0017 $\pm$ 0.0002	0.0013 $\pm$ 0.0001	0.0046 $\pm$ 0.0006
500	0.0017 $\pm$ 0.0005	0.0014 $\pm$ 0.0003	0.0050 $\pm$ 0.0009
600	0.0029 $\pm$ 0.0015	0.0023 $\pm$ 0.0011	0.0082 $\pm$ 0.0016
700	0.0033 $\pm$ 0.0009	0.0033 $\pm$ 0.0014	0.0111 $\pm$ 0.0036
800	0.0041 $\pm$ 0.0011	0.0036 $\pm$ 0.0008	0.0144 $\pm$ 0.0032
900	0.0050 $\pm$ 0.0009	0.0045 $\pm$ 0.0009	0.0177 $\pm$ 0.0040

Considerações Finais

- O ganho assintótico teórico do BMSSP ($O(m \log^{2/3} n)$) é um feito matemático excepcional (Duan et al.).
- Contudo, observamos no experimento (via Profiling) que o tempo prático é severamente influenciado pelas chamadas recursivas, alocação excessiva da matriz e operações de amostragem.
- O **Dijkstra Clássico** aliado a um Min-Heap nativo com manipulação eficiente de ponteiros exibe *overhead* prático tão baixo que costuma vencer em tempo real no hardware moderno em cenários sub-quadráticos.
- O ambiente de CI/CD via `Makefile` permite rápida replicação, colaboração inter-equipes fluida e geração determinística da documentação científica.

Obrigado!

Dúvidas?